REPORT OF SCHEMA AND RELATIONSHIPS

(Data Analyst Internship)

Name:	TANVI AGGARWAL
Project Title:	Sales and Analytics System – Salase Industries
Duration:	23rd June 2025 – 25th June 2025
Focus:	System Design & Relational Database Development
Domain:	Finance Industry
Tool Used:	MySQL Workbench
Language:	SQL

TABLE OF CONTENT

1. Introduction
2. Schema Overview
3. Tables and Fields
    - Customers
    - Products
    - Sales
    - SalesAgents
    - Agent_Sales
4. Relationships
5. SQL Data Analysis Questions
    - Aggregation Functions (SUM, COUNT, AVG, MAX, MIN)
    - WHERE Clause
    - HAVING Clause
    - ORDER BY Clause
    - LIMIT Clause
6. Conclusion

INTRODUCTION

This project titled **"Sales and Analytics System – Salase Industries"** is developed as part of my Data Analytics internship under the guidance of **SkilledUp**, aimed at building a practical understanding of real-world business data systems using **SQL**. **Salase Industries** is a company operating in the **Fast-Moving Consumer Goods (FMCG)** sector, dealing with high-volume and fast-paced sales. Like many businesses in this domain, the company needed a comprehensive system to manage and analyze its growing data related to **customers, products, sales, and sales agents**.

The goal of this project was to design a robust **Sales and Analytics System** that could not only store structured data efficiently but also help in answering complex business queries through data analysis. The system uses relational database tables to capture:

- Customer details and geography
- Product categories, pricing, and stock levels
- Sales transactions including amounts, quantities, and dates
- Agent performance data tied to sales activities

Through this project, I implemented and executed **30 practical SQL queries** that addressed common business challenges and decision-making needs, such as:

- Calculating total and category-wise revenue
- Identifying top-selling products and high-value customers
- Monitoring inventory levels and restocking needs
- Evaluating agent-wise sales contribution and effectiveness
- Tracking recent transactions and time-based trends

In addition to technical SQL skills, this project strengthened my ability to **translate business questions into data queries**, and more importantly, convert raw data into **actionable insights** that support informed business strategies.

This system serves as a foundational model for any growing business in the FMCG space to become more **data-driven**, efficient, and customer-focused.

SCHEMA OVERVIEW

The database schema is designed to support efficient tracking and analysis of sales, customers, products, and sales agents. It includes **five core tables**, structured in a relational format:

- **1. Customers**

Stores customer details.

- CustomerID, Name, Email, Phone, City, State, JoinDate

- **2. Products**

Contains product information.

- ProductID, ProductName, Category, UnitPrice, StockQuantity

- **3. Sales**

Records each sales transaction.

- SaleID, CustomerID, ProductID, SaleDate, QuantitySold, TotalAmount

- **4. SalesAgents**

Tracks sales agents' data.

- AgentID, AgentName, Region, Phone, JoiningDate

- **5. Agent_Sales**

Links agents to the sales they handled (many-to-many relationship).

- SaleID, AgentID

This structure allows for comprehensive business analysis, such as tracking revenue, identifying top products, evaluating agent performance, and understanding customer behavior—essential for data-driven decisions in the FMCG industry.

TABLES

- **USERS**

It is used to store and manage detailed information about each person who uses the system—whether for creating accounts, taking loans, making transactions, or investing money.

Column Name	Data Type	Constraints
CustomerID	INT	PRIMARY KEY, NOT NULL
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NULL
Phone	VARCHAR(15)	NULL
City	VARCHAR(50)	NULL
State	VARCHAR(50)	NULL
JoinDate	DATE	NULL

- **PRODUCTS**

It is responsible for storing and managing details about each financial account owned by users. It plays a vital role in tracking balances, account types, and linking transactions to specific users.

Column Name	Data Type	Constraints
ProductId	INT	PRIMARY KEY, NOT NULL
ProductName	VARCHAR(100)	NOT NULL
Category	VARCHAR(50)	NULL
UnitPrice	DECIMAL(10,2)	NULL
StockQuantity	INT	NULL

SALES

It manages all system administrators who are responsible for overseeing user activity, managing records, and maintaining the system.

Column Name	Data Type	Constraints
SaleID	INT	PRIMARY KEY, NOT NULL
CustomerID	INT	NULL
ProductID	INT	NULL
SaleDate	DATE	NULL
QuantitySold	INT	NULL
TotalAmount	DECIMAL(10,2)	NULL

- **SalesAgents**

  Tracks and manages user investments in financial products like mutual funds, fixed deposits, stocks, etc.

Column Name	Data Type	Constraints
AgentID	INT	PRIMARY KEY, NOT NULL
AgentName	VARCHAR(100)	NULL
Region	VARCHAR(50)	NULL
Phone	DECIMAL(15)	NULL
JoiningDate	DATE	NULL

- **Agent_Sales**

Stores all **loan records** applied for or approved by users.

Column Name	Data Type	Constraints
SaleID	INT	PRIMARY KEY, NOT NULL
AgentID	INT	PRIMARY KEY , NOT NULL

RELATIONSHIPS

In this project, we created 5 main tables to store different types of information for the business. Each table stores data about one important part of the system — like customers, products, sales, and agents. Now let's see how they are all **connected**.

1. Customers and Sales

- The **Customers** table stores details of all people who buy products.
- The **Sales** table stores each purchase or sale made.

One customer can make many purchases, so:

- **One customer → many sales**

  This is called a **One-to-Many relationship**.

Example:

If "Anjali Sharma" is a customer, she may have bought 5 times. Each of those purchases will be a row in the **Sales** table with her CustomerID.

2. Products and Sales

- The **Products** table has details about each product (like name, price, and category).
- The **Sales** table keeps a record of which product was sold in each sale.

One product can be sold many times, so:

- **One product → many sales**

  Again, this is a **One-to-Many relationship**.

Example:

If the product "Chocolate Bar" is popular, it might appear in 50 different sales.

3. Sales and Sales Agents (via Agent_Sales)

- The **SalesAgents** table stores info about all agents who handle sales.

- But here's the twist: a **single sale** can be handled by **more than one agent**, and one agent can handle **many different sales**.

So, this is a **Many-to-Many relationship**, and for that, we use a middle table called **Agent_Sales**.

- The **Agent_Sales** table connects:
    - SaleID (from Sales table)
    - AgentID (from SalesAgents table)

This means:

- **One sale → can have multiple agents**
- **One agent → can handle multiple sales**

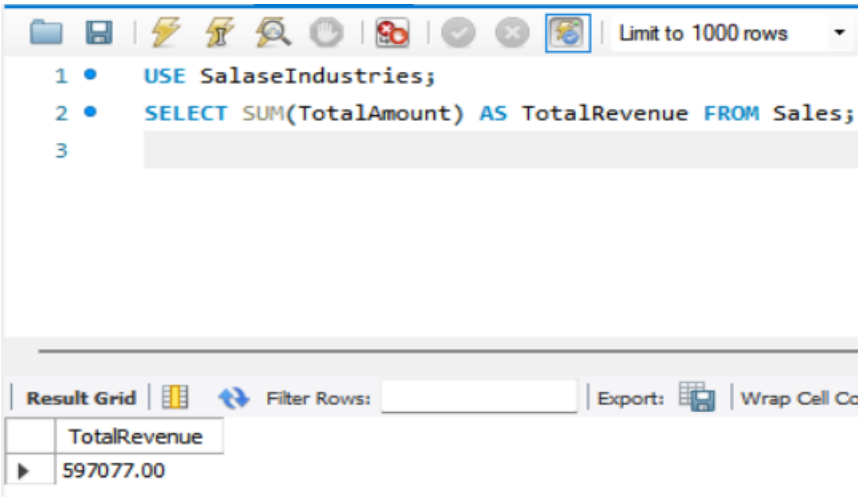
This **middle table** helps us track who handled which sale.

Example:
Sale ID 101 might be handled by Agent A and Agent B. Agent A might also handle Sale ID 102, 105, etc

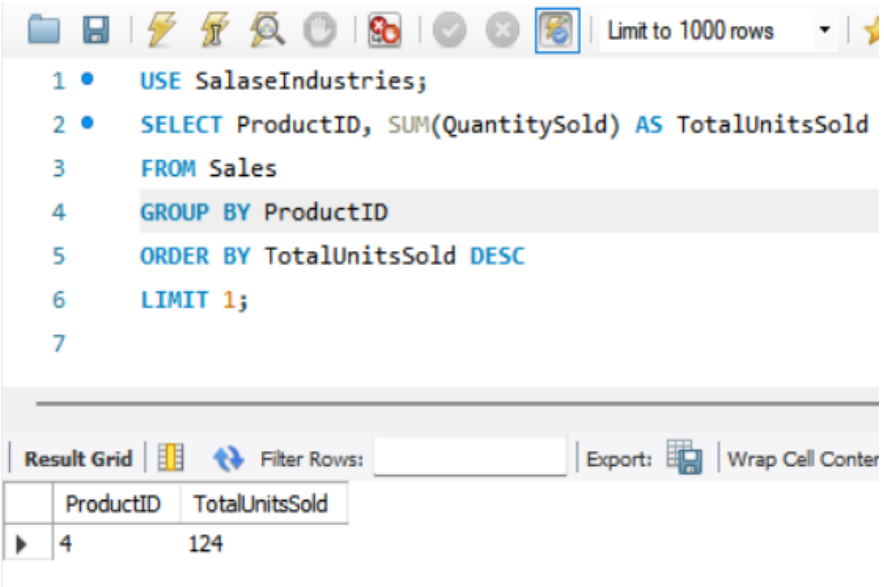
SQL DATA ANALYSIS

❖Aggregation function (sum , count , avg , max , min)

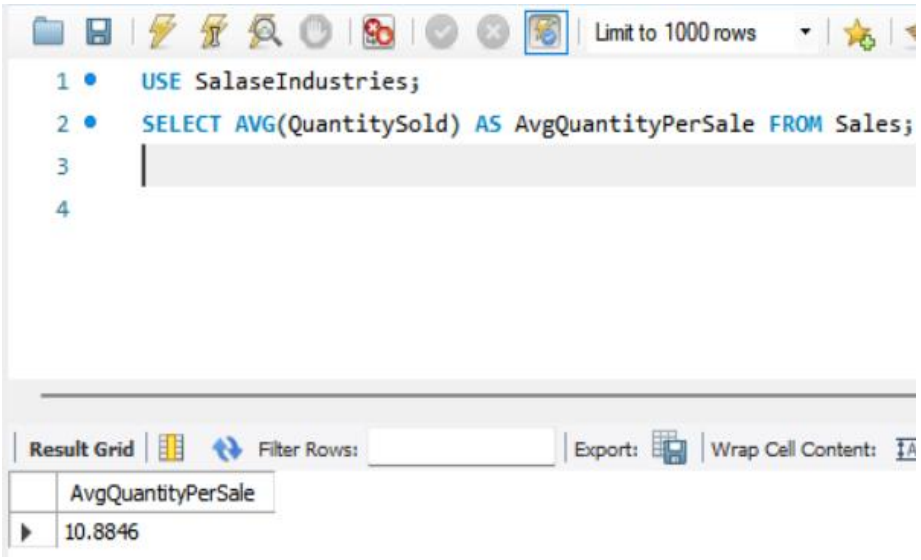
1. **What is the total sales revenue generated across all products?**



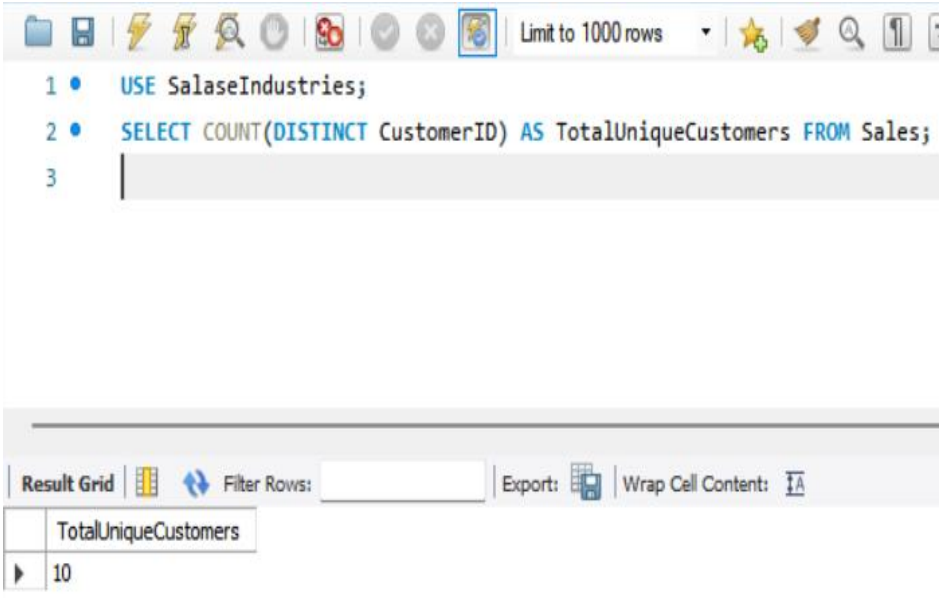
2. **Which product has the highest number of units sold?**



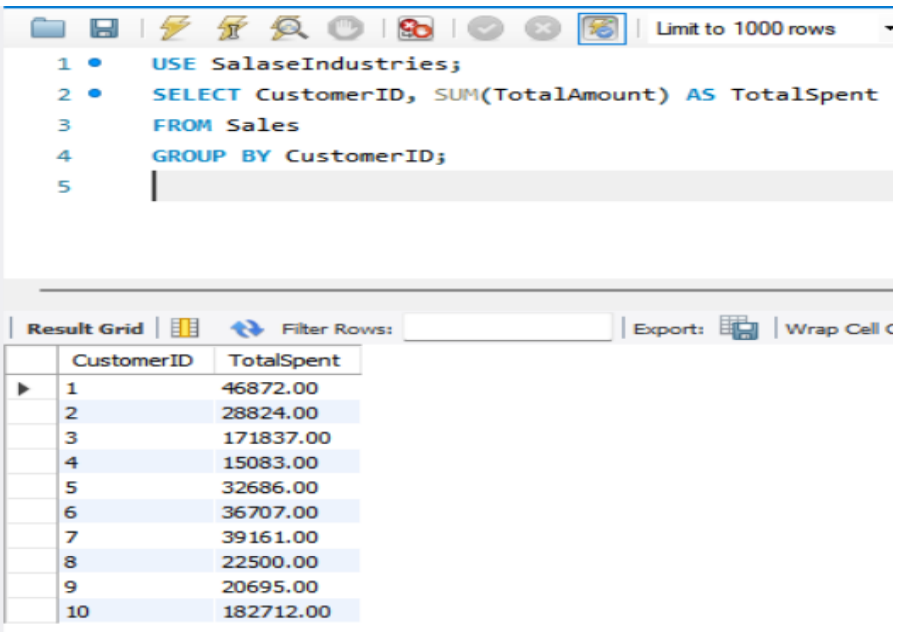
3. **What is the average quantity of products sold per sale?**



4. **What is the total number of customers who have made purchases?**



5. **What is the total amount spent by each customer?**

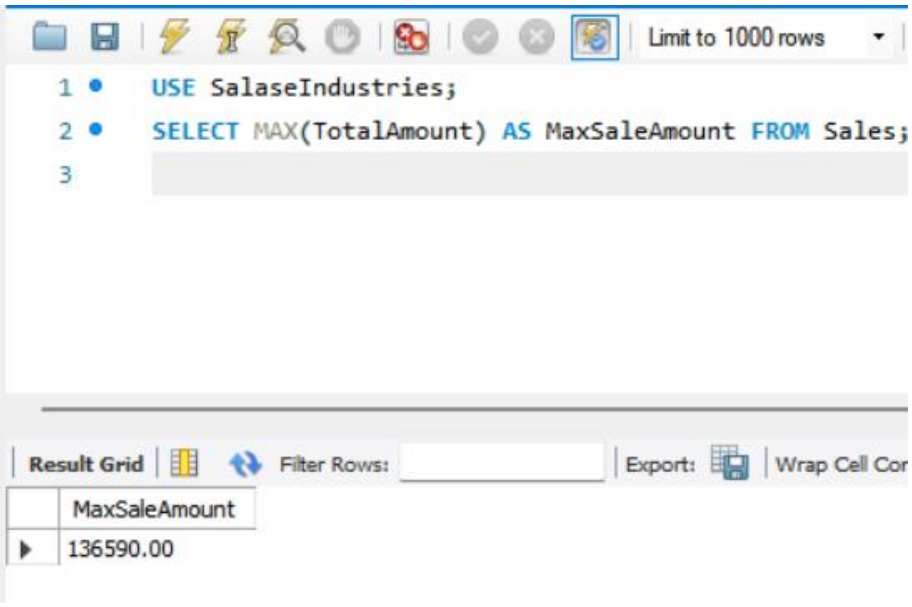


```
USE SalaseIndustries;
SELECT CustomerID, SUM(TotalAmount) AS TotalSpent
FROM Sales
GROUP BY CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

CustomerID	TotalSpent
1	46872.00
2	28824.00
3	171837.00
4	15083.00
5	32686.00
6	36707.00
7	39161.00
8	22500.00
9	20695.00
10	182712.00

6. **What is the maximum sale amount recorded?**

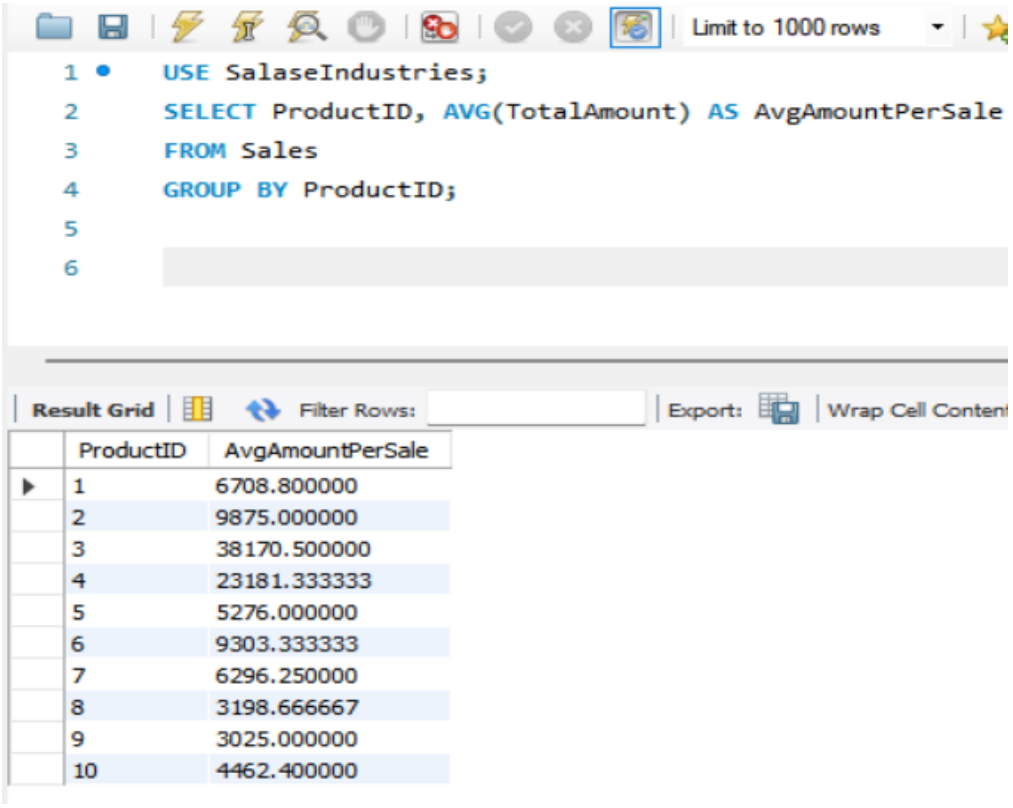


```
USE SalaseIndustries;
SELECT MAX(TotalAmount) AS MaxSaleAmount FROM Sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

MaxSaleAmount
136590.00

7. **What is the average total amount per sale for each product?**

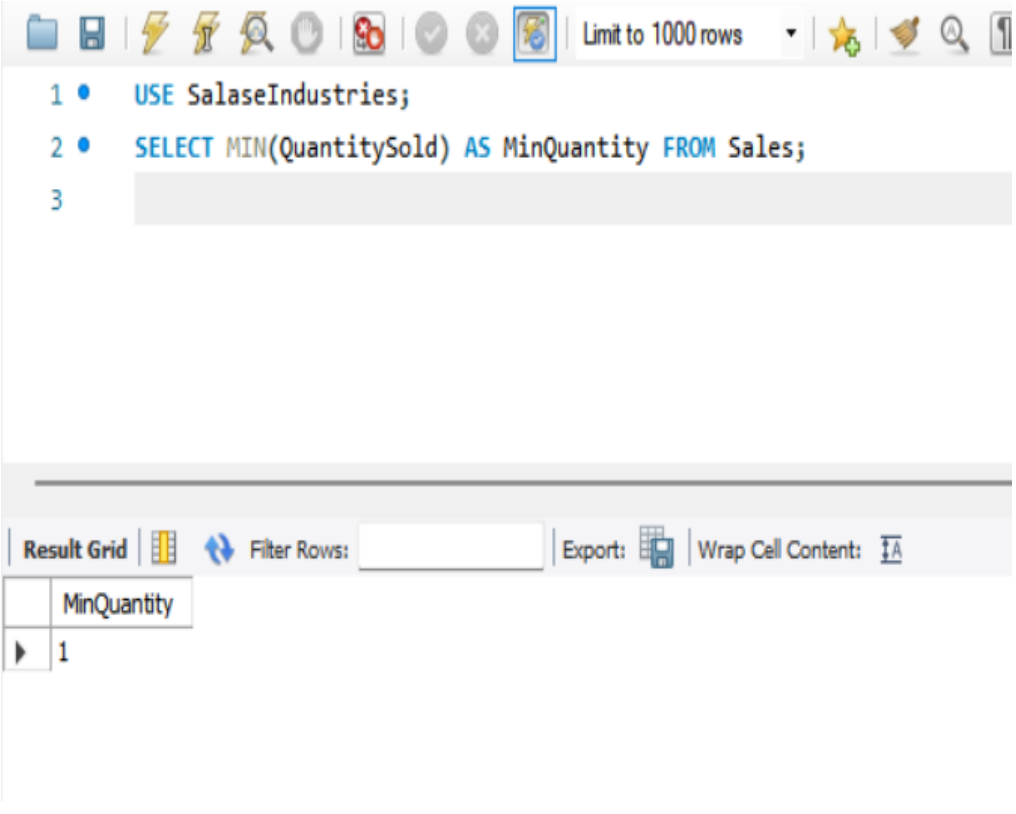


```
USE SalaseIndustries;
SELECT ProductID, AVG(TotalAmount) AS AvgAmountPerSale
FROM Sales
GROUP BY ProductID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

ProductID	AvgAmountPerSale
1	6708.800000
2	9875.000000
3	38170.500000
4	23181.333333
5	5276.000000
6	9303.333333
7	6296.250000
8	3198.666667
9	3025.000000
10	4462.400000

8. **What is the minimum quantity sold in a single transaction?**

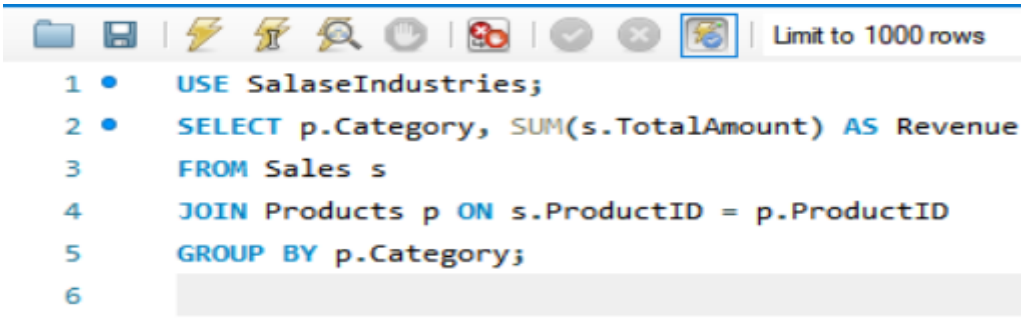


```
USE SalaseIndustries;
SELECT MIN(QuantitySold) AS MinQuantity FROM Sales;
```

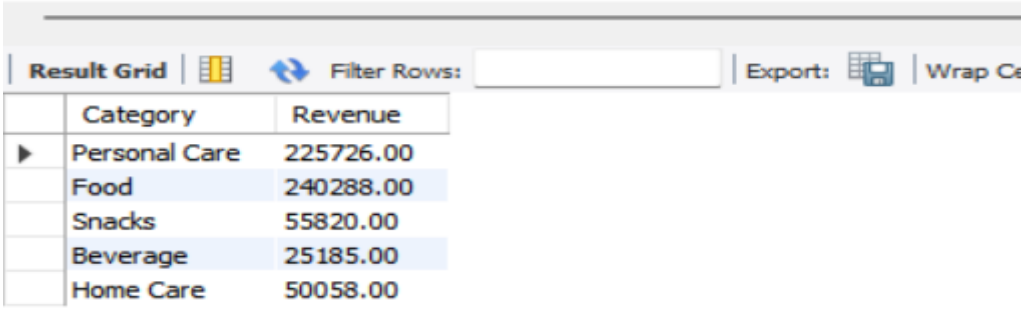
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

MinQuantity
1

9. **What is the total revenue generated per product category?**



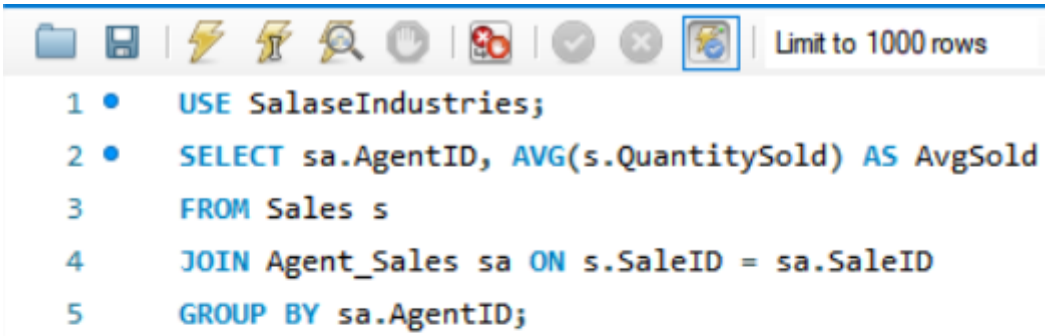
```
Limit to 1000 rows
1  USE SalaseIndustries;
2  SELECT p.Category, SUM(s.TotalAmount) AS Revenue
3  FROM Sales s
4  JOIN Products p ON s.ProductID = p.ProductID
5  GROUP BY p.Category;
6
```



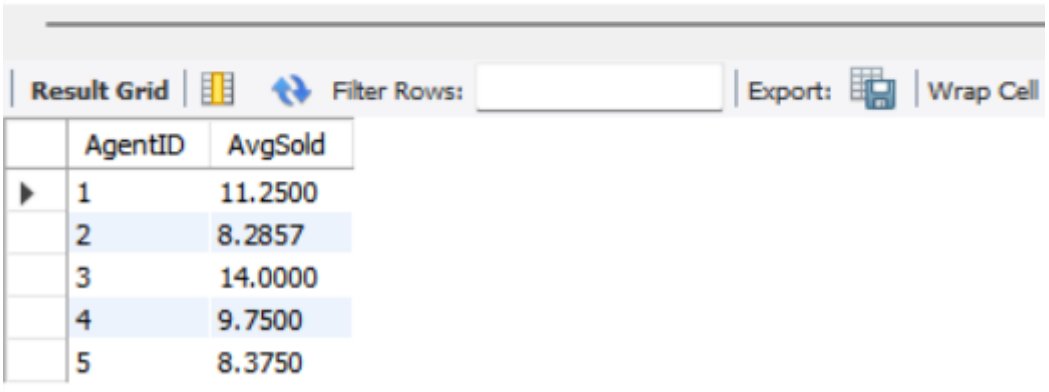
Result Grid | Filter Rows: | Export: | Wrap Ce

Category	Revenue
Personal Care	225726.00
Food	240288.00
Snacks	55820.00
Beverage	25185.00
Home Care	50058.00

10. **What is the average number of products sold by each sales agent?**



```
Limit to 1000 rows
1  USE SalaseIndustries;
2  SELECT sa.AgentID, AVG(s.QuantitySold) AS AvgSold
3  FROM Sales s
4  JOIN Agent_Sales sa ON s.SaleID = sa.SaleID
5  GROUP BY sa.AgentID;
6
```

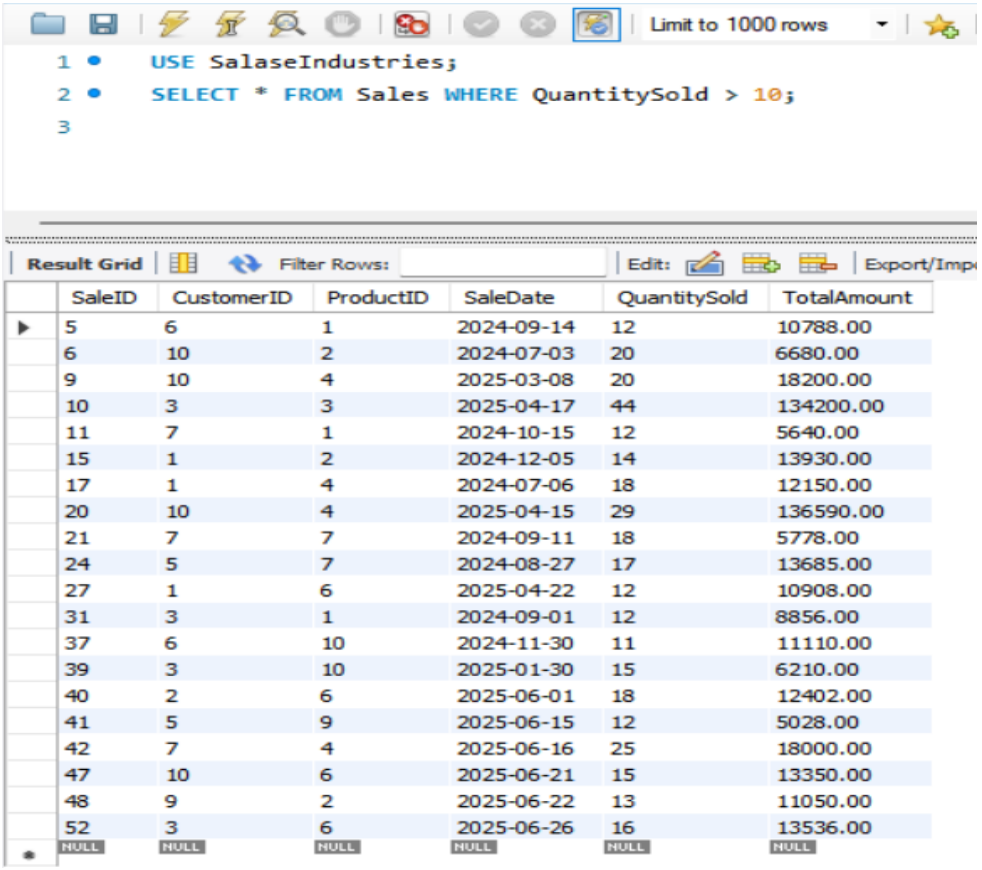


Result Grid | Filter Rows: | Export: | Wrap Cell

AgentID	AvgSold
1	11.2500
2	8.2857
3	14.0000
4	9.7500
5	8.3750

❖ WHERE Clause

11. List all sales where the quantity sold is greater than 10.

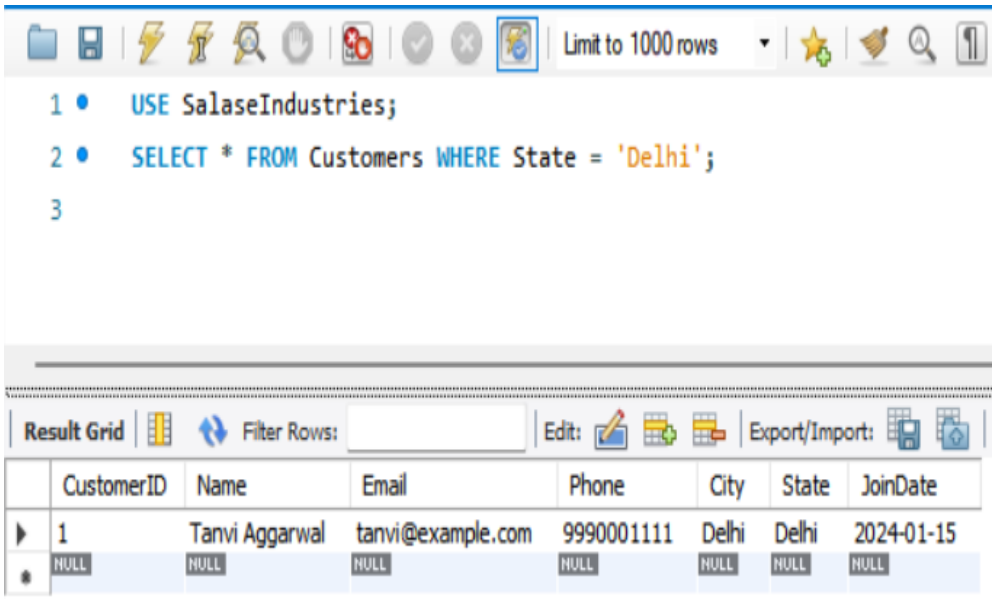


```
USE SalaseIndustries;
SELECT * FROM Sales WHERE QuantitySold > 10;
```

Result Grid

SaleID	CustomerID	ProductID	SaleDate	QuantitySold	TotalAmount
5	6	1	2024-09-14	12	10788.00
6	10	2	2024-07-03	20	6680.00
9	10	4	2025-03-08	20	18200.00
10	3	3	2025-04-17	44	134200.00
11	7	1	2024-10-15	12	5640.00
15	1	2	2024-12-05	14	13930.00
17	1	4	2024-07-06	18	12150.00
20	10	4	2025-04-15	29	136590.00
21	7	7	2024-09-11	18	5778.00
24	5	7	2024-08-27	17	13685.00
27	1	6	2025-04-22	12	10908.00
31	3	1	2024-09-01	12	8856.00
37	6	10	2024-11-30	11	11110.00
39	3	10	2025-01-30	15	6210.00
40	2	6	2025-06-01	18	12402.00
41	5	9	2025-06-15	12	5028.00
42	7	4	2025-06-16	25	18000.00
47	10	6	2025-06-21	15	13350.00
48	9	2	2025-06-22	13	11050.00
52	3	6	2025-06-26	16	13536.00
NULL	NULL	NULL	NULL	NULL	NULL

12. Find all customers from the state of "Delhi".

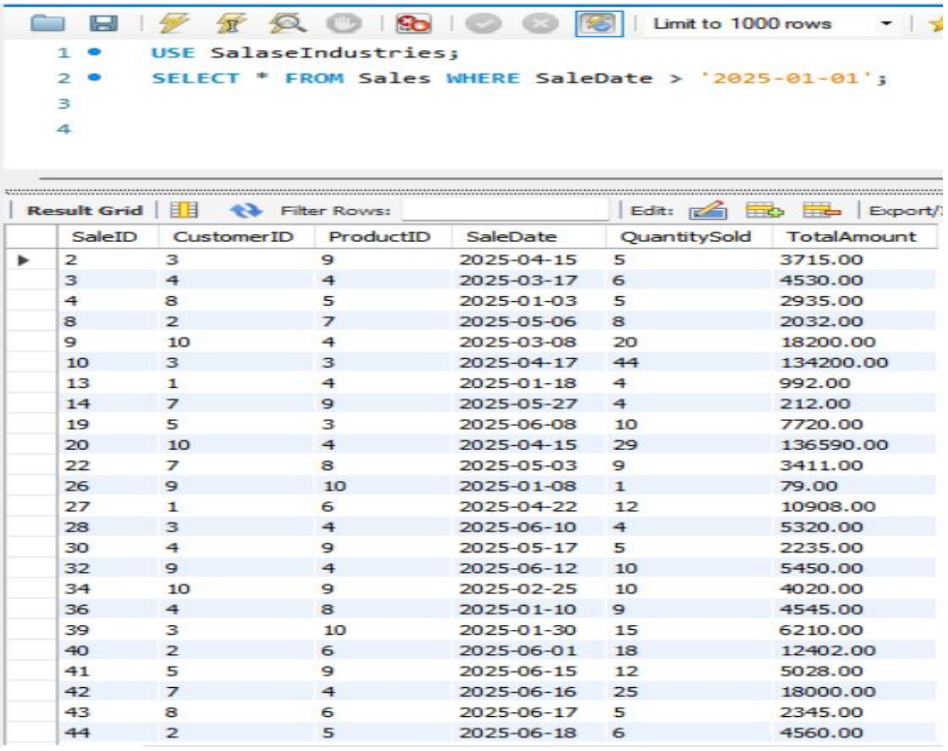


```
USE SalaseIndustries;
SELECT * FROM Customers WHERE State = 'Delhi';
```

Result Grid

CustomerID	Name	Email	Phone	City	State	JoinDate
1	Tanvi Aggarwal	tanvi@example.com	9990001111	Delhi	Delhi	2024-01-15
NULL	NULL	NULL	NULL	NULL	NULL	NULL

13. Retrieve all sales made after January 1, 2025.

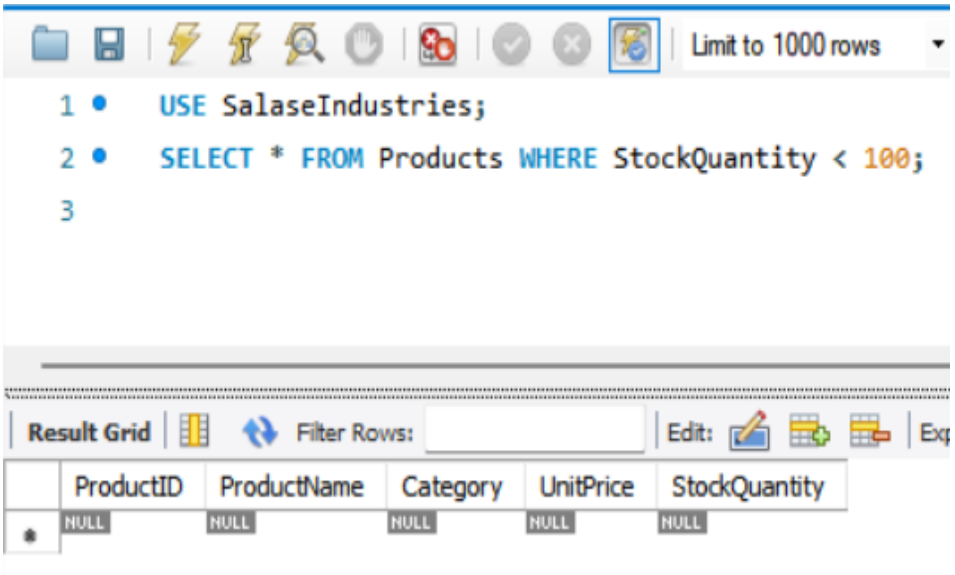


```
1  USE SalaseIndustries;
2  SELECT * FROM Sales WHERE SaleDate > '2025-01-01';
3
4
```

Result Grid | Filter Rows: | Edit: | Export/

SaleID	CustomerID	ProductID	SaleDate	QuantitySold	TotalAmount
2	3	9	2025-04-15	5	3715.00
3	4	4	2025-03-17	6	4530.00
4	8	5	2025-01-03	5	2935.00
8	2	7	2025-05-06	8	2032.00
9	10	4	2025-03-08	20	18200.00
10	3	3	2025-04-17	44	134200.00
13	1	4	2025-01-18	4	992.00
14	7	9	2025-05-27	4	212.00
19	5	3	2025-06-08	10	7720.00
20	10	4	2025-04-15	29	136590.00
22	7	8	2025-05-03	9	3411.00
26	9	10	2025-01-08	1	79.00
27	1	6	2025-04-22	12	10908.00
28	3	4	2025-06-10	4	5320.00
30	4	9	2025-05-17	5	2235.00
32	9	4	2025-06-12	10	5450.00
34	10	9	2025-02-25	10	4020.00
36	4	8	2025-01-10	9	4545.00
39	3	10	2025-01-30	15	6210.00
40	2	6	2025-06-01	18	12402.00
41	5	9	2025-06-15	12	5028.00
42	7	4	2025-06-16	25	18000.00
43	8	6	2025-06-17	5	2345.00
44	2	5	2025-06-18	6	4560.00

14. Find all products with stock quantity less than 100.



```
1  USE SalaseIndustries;
2  SELECT * FROM Products WHERE StockQuantity < 100;
3
```

Result Grid | Filter Rows: | Edit: | Exp

ProductID	ProductName	Category	UnitPrice	StockQuantity
NULL	NULL	NULL	NULL	NULL

15. Get details of customers who joined in 2024.

```
1 • USE SalaseIndustries;
2 • SELECT * FROM Customers WHERE YEAR(JoinDate) = 2024;
3
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

CustomerID	Name	Email	Phone	City	State	JoinDate
1	Tanvi Aggarwal	tanvi@example.com	9990001111	Delhi	Delhi	2024-01-15
2	Ravi Sharma	ravi@example.com	9988001122	Mumbai	Maharashtra	2024-02-10
5	Meera Jain	meera@example.com	9988776655	Chennai	Tamil Nadu	2024-06-10
7	Pooja Das	pooja@example.com	9344119988	Kolkata	West Bengal	2024-04-25
9	Simran Kaur	simran@example.com	9223334445	Amritsar	Punjab	2024-09-12
10	Vikas Rawat	vikas@example.com	9112233445	Dehradun	Uttarakhand	2024-07-08
NULL	NULL	NULL	NULL	NULL	NULL	NULL

❖ HAVING Clause

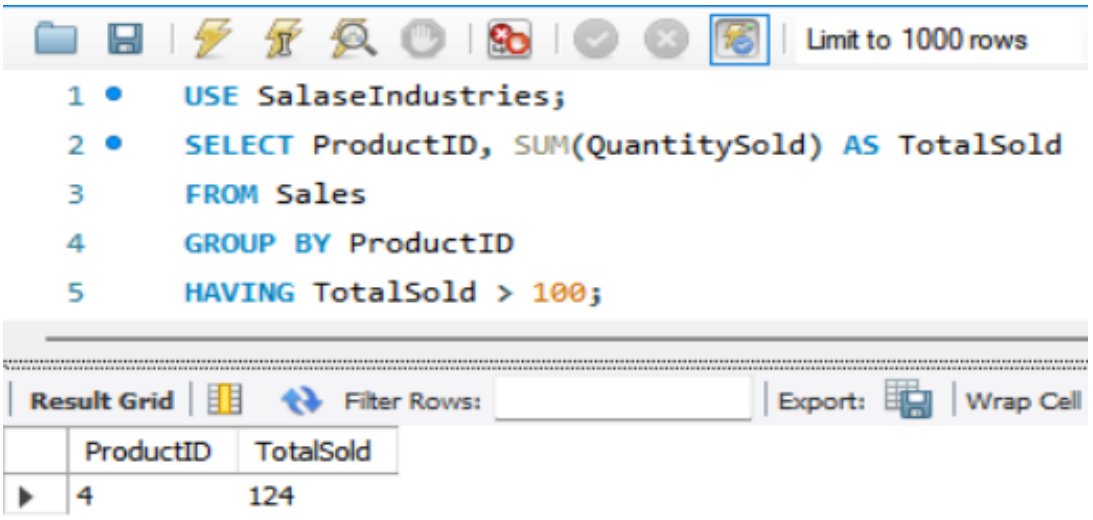
16. Which customers have spent more than ₹50,000?

```
1 • USE SalaseIndustries;
2 • SELECT CustomerID, SUM(TotalAmount) AS TotalSpent
3   FROM Sales
4   GROUP BY CustomerID
5   HAVING TotalSpent > 50000;
```

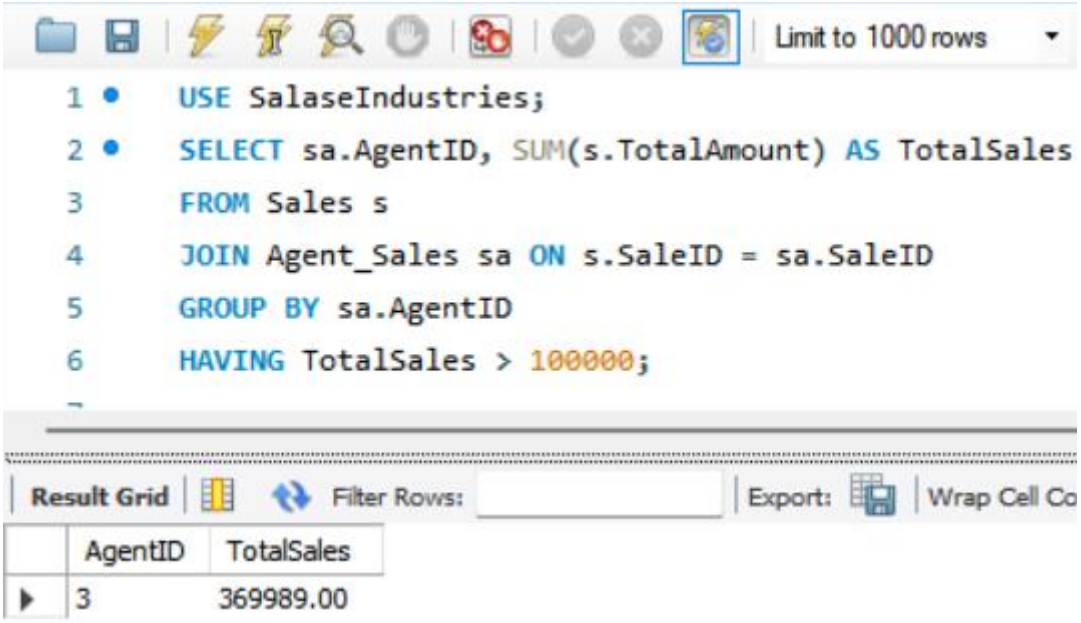
Result Grid | Filter Rows: | Export: | Wrap Cell Co

CustomerID	TotalSpent
3	171837.00
10	182712.00

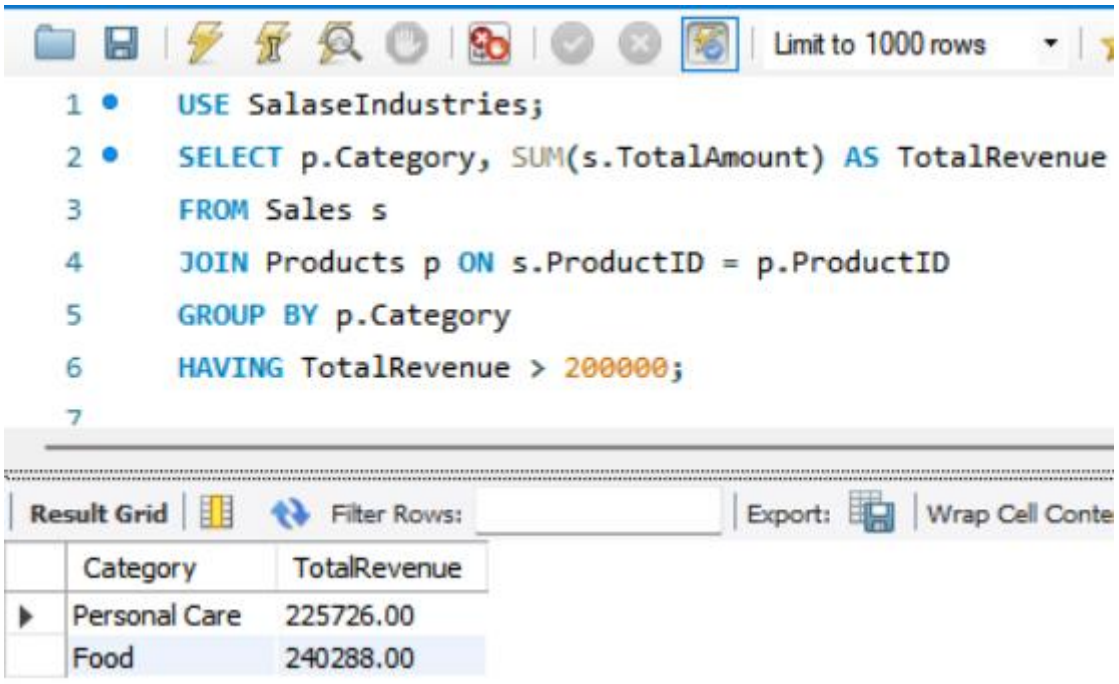
17. List products that have sold more than 100 units.



18. Identify sales agents who have handled sales worth more than ₹1,00,000.



19. Find product categories that generated revenue over ₹2,00,000.

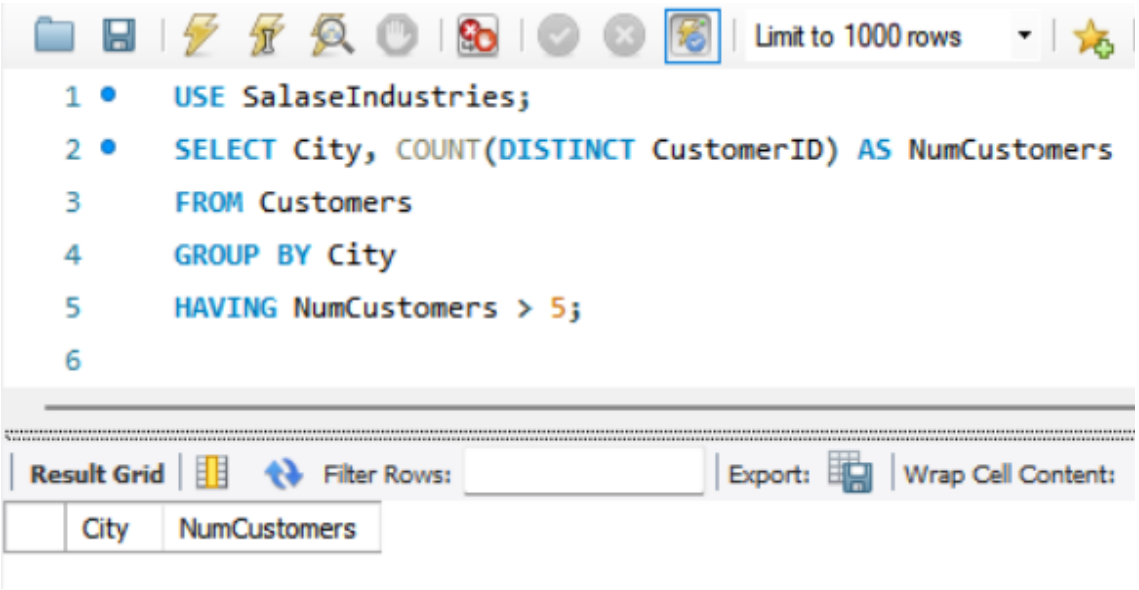


```
USE SalaseIndustries;
SELECT p.Category, SUM(s.TotalAmount) AS TotalRevenue
FROM Sales s
JOIN Products p ON s.ProductID = p.ProductID
GROUP BY p.Category
HAVING TotalRevenue > 200000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

Category	TotalRevenue
Personal Care	225726.00
Food	240288.00

20. Show cities with more than 5 unique customers.



```
USE SalaseIndustries;
SELECT City, COUNT(DISTINCT CustomerID) AS NumCustomers
FROM Customers
GROUP BY City
HAVING NumCustomers > 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

City	NumCustomers

❖ORDER BY Clause

21. List all products ordered by unit price (high to low).

```
1  USE SalaseIndustries;
2  SELECT * FROM Products ORDER BY UnitPrice DESC;
3
4
```

Result Grid | Filter Rows: | Edit:

ProductID	ProductName	Category	UnitPrice	StockQuantity
5	Atta 5kg	Food	250.00	150
2	Shampoo	Personal Care	150.00	100
8	Detergent 1kg	Home Care	120.00	180
9	Floor Cleaner	Home Care	90.00	160
10	Toilet Cleaner	Home Care	80.00	130
4	Rice 1kg	Food	60.00	300
1	Toothpaste	Personal Care	45.00	200
7	Cold Drink	Beverage	40.00	300
3	Soap	Personal Care	30.00	250
6	Biscuits	Snacks	20.00	400
NULL	NULL	NULL	NULL	NULL

22. Show top 10 customers by total amount spent.

```
1  USE SalaseIndustries;
2  SELECT CustomerID, SUM(TotalAmount) AS TotalSpent
3  FROM Sales
4  GROUP BY CustomerID
5  ORDER BY TotalSpent DESC
6  LIMIT 10;
7
```

Result Grid | Filter Rows: | Export: | Wrap Cell

CustomerID	TotalSpent
10	182712.00
3	171837.00
1	46872.00
7	39161.00
6	36707.00
5	32686.00
2	28824.00
8	22500.00
9	20695.00
4	15083.00

23. List all sales sorted by date, most recent first.

```
USE SalaseIndustries;
SELECT * FROM Sales ORDER BY SaleDate DESC;
```

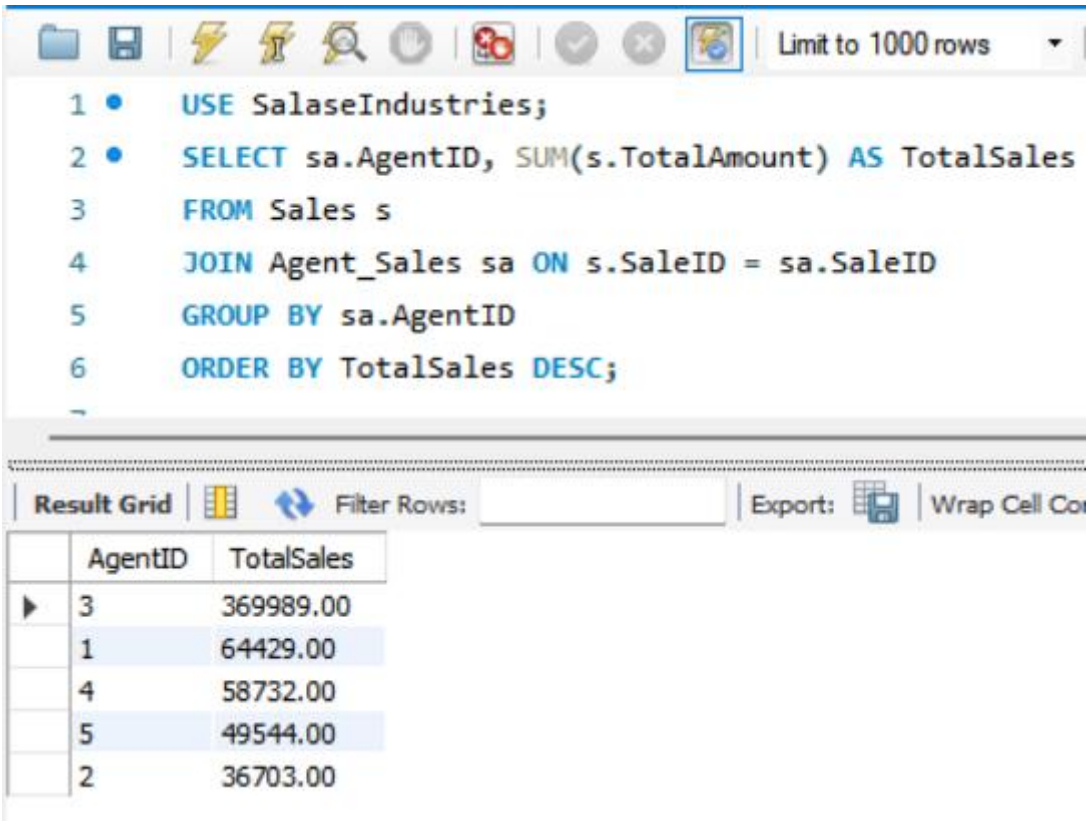
Result Grid | Filter Rows: | Edit: | Export/Impo

SaleID	CustomerID	ProductID	SaleDate	QuantitySold	TotalAmount
52	3	6	2025-06-26	16	13536.00
51	8	1	2025-06-25	8	4144.00
50	7	5	2025-06-24	9	6120.00
49	6	2	2025-06-23	10	7840.00
48	9	2	2025-06-22	13	11050.00
47	10	6	2025-06-21	15	13350.00
46	5	8	2025-06-20	4	1640.00
45	4	10	2025-06-19	7	3773.00
44	2	5	2025-06-18	6	4560.00
43	8	6	2025-06-17	5	2345.00
42	7	4	2025-06-16	25	18000.00
41	5	9	2025-06-15	12	5028.00
32	9	4	2025-06-12	10	5450.00
28	3	4	2025-06-10	4	5320.00
19	5	3	2025-06-08	10	7720.00
40	2	6	2025-06-01	18	12402.00
14	7	9	2025-05-27	4	212.00
30	4	9	2025-05-17	5	2235.00
8	2	7	2025-05-06	8	2032.00
22	7	8	2025-05-03	9	3411.00
27	1	6	2025-04-22	12	10908.00
10	3	3	2025-04-17	44	134200.00
20	10	4	2025-04-15	29	136590.00
2	3	9	2025-04-15	5	3715.00

Result Grid | Filter Rows: | Edit: | Export/I

SaleID	CustomerID	ProductID	SaleDate	QuantitySold	TotalAmount
36	4	8	2025-01-10	9	4545.00
26	9	10	2025-01-08	1	79.00
4	8	5	2025-01-03	5	2935.00
38	9	1	2024-12-28	6	4116.00
35	1	5	2024-12-25	6	7752.00
15	1	2	2024-12-05	14	13930.00
23	1	10	2024-12-02	10	1140.00
37	6	10	2024-11-30	11	11110.00
18	6	7	2024-11-23	5	3690.00
16	8	5	2024-11-20	6	5676.00
12	10	3	2024-11-19	8	3872.00
7	8	4	2024-11-18	8	7400.00
1	2	9	2024-11-15	5	2940.00
11	7	1	2024-10-15	12	5640.00
33	2	3	2024-10-05	10	6890.00
25	5	5	2024-09-29	7	4613.00
5	6	1	2024-09-14	12	10788.00
21	7	7	2024-09-11	18	5778.00
31	3	1	2024-09-01	12	8856.00
24	5	7	2024-08-27	17	13685.00
29	6	6	2024-08-20	3	3279.00
17	1	4	2024-07-06	18	12150.00
6	10	2	2024-07-03	20	6680.00
NULL	NULL	NULL	NULL	NULL	NULL

24. List sales agents ordered by total sales handled.

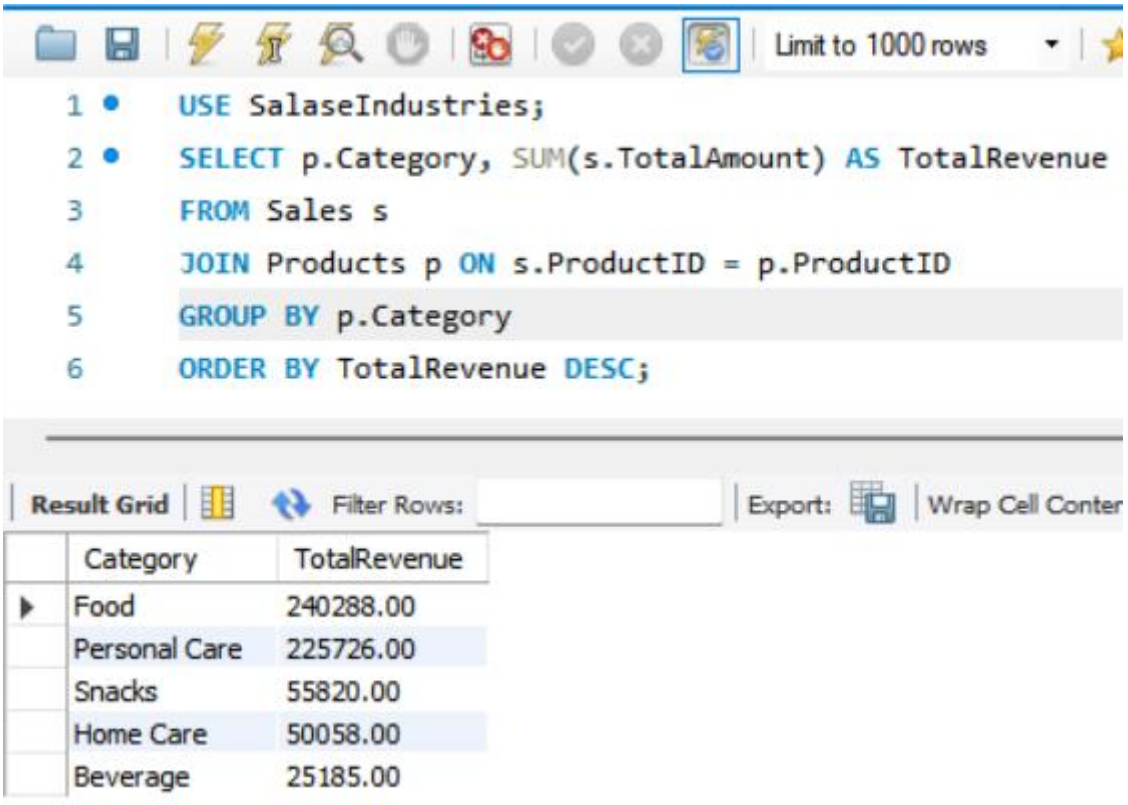


```
1  USE SalaseIndustries;
2  SELECT sa.AgentID, SUM(s.TotalAmount) AS TotalSales
3  FROM Sales s
4  JOIN Agent_Sales sa ON s.SaleID = sa.SaleID
5  GROUP BY sa.AgentID
6  ORDER BY TotalSales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

AgentID	TotalSales
3	369989.00
1	64429.00
4	58732.00
5	49544.00
2	36703.00

25. Show product categories sorted by total revenue.



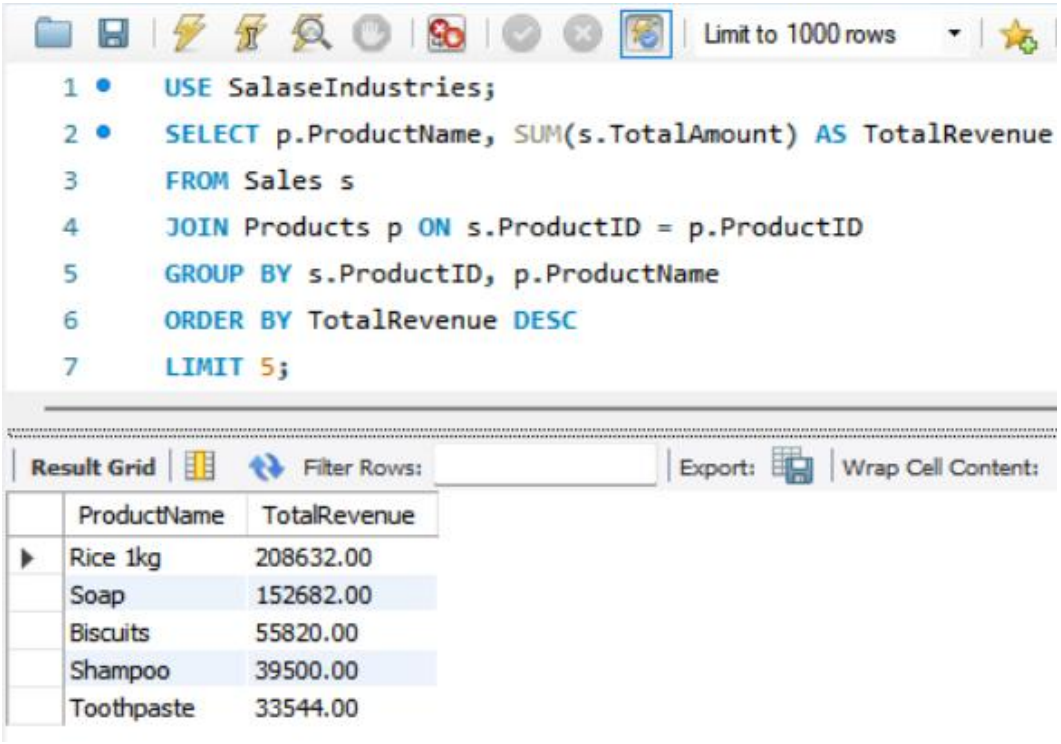
```
1  USE SalaseIndustries;
2  SELECT p.Category, SUM(s.TotalAmount) AS TotalRevenue
3  FROM Sales s
4  JOIN Products p ON s.ProductID = p.ProductID
5  GROUP BY p.Category
6  ORDER BY TotalRevenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

Category	TotalRevenue
Food	240288.00
Personal Care	225726.00
Snacks	55820.00
Home Care	50058.00
Beverage	25185.00

❖LIMIT Clause

26. Retrieve the top 5 products with the highest revenue.

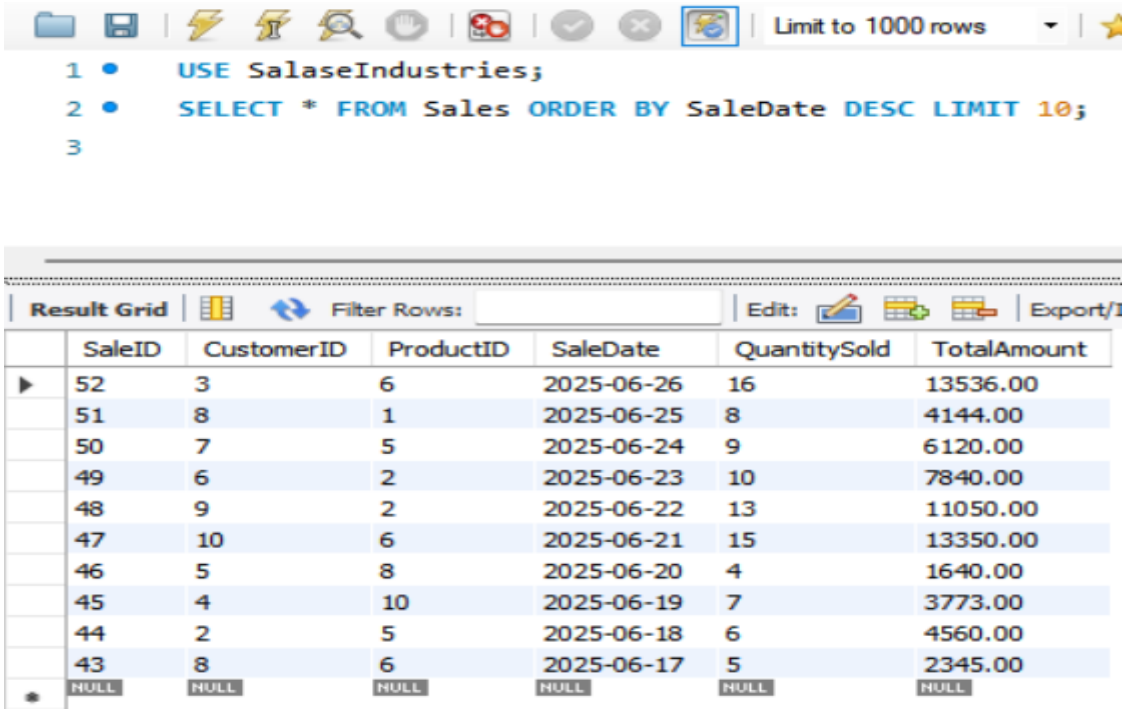


```
USE SalaseIndustries;
SELECT p.ProductName, SUM(s.TotalAmount) AS TotalRevenue
FROM Sales s
JOIN Products p ON s.ProductID = p.ProductID
GROUP BY s.ProductID, p.ProductName
ORDER BY TotalRevenue DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	TotalRevenue
Rice 1kg	208632.00
Soap	152682.00
Biscuits	55820.00
Shampoo	39500.00
Toothpaste	33544.00

27. Display the 10 most recent sales transactions.

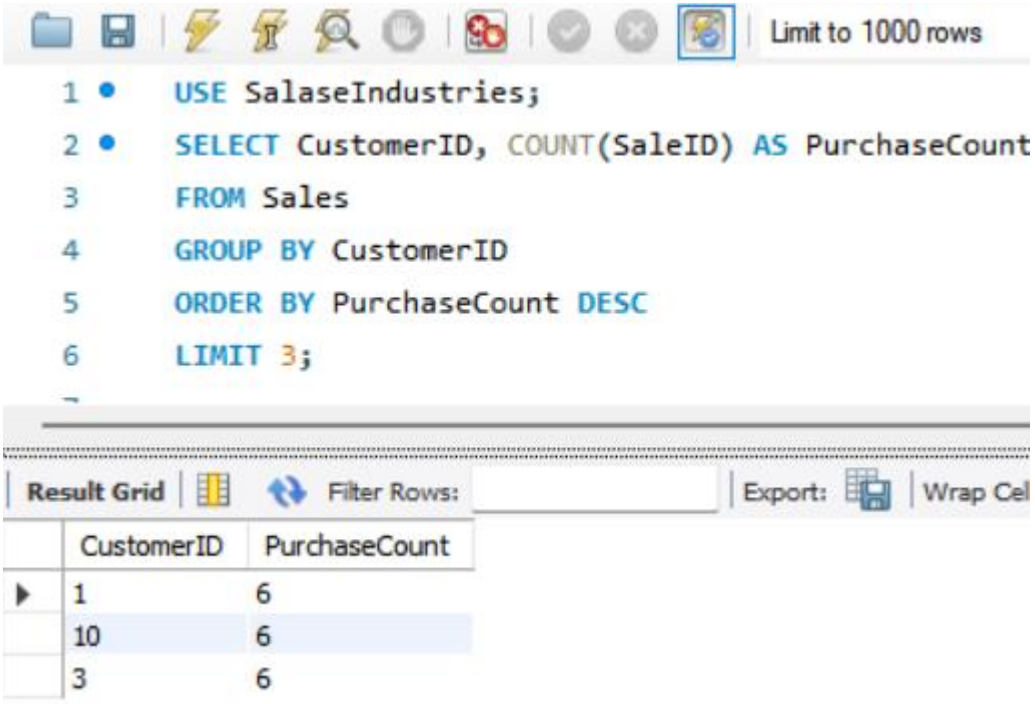


```
USE SalaseIndustries;
SELECT * FROM Sales ORDER BY SaleDate DESC LIMIT 10;
```

Result Grid | Filter Rows: | Edit: | Export/

SaleID	CustomerID	ProductID	SaleDate	QuantitySold	TotalAmount
52	3	6	2025-06-26	16	13536.00
51	8	1	2025-06-25	8	4144.00
50	7	5	2025-06-24	9	6120.00
49	6	2	2025-06-23	10	7840.00
48	9	2	2025-06-22	13	11050.00
47	10	6	2025-06-21	15	13350.00
46	5	8	2025-06-20	4	1640.00
45	4	10	2025-06-19	7	3773.00
44	2	5	2025-06-18	6	4560.00
43	8	6	2025-06-17	5	2345.00
NULL	NULL	NULL	NULL	NULL	NULL

28. Get the top 3 customers with the highest number of purchases.

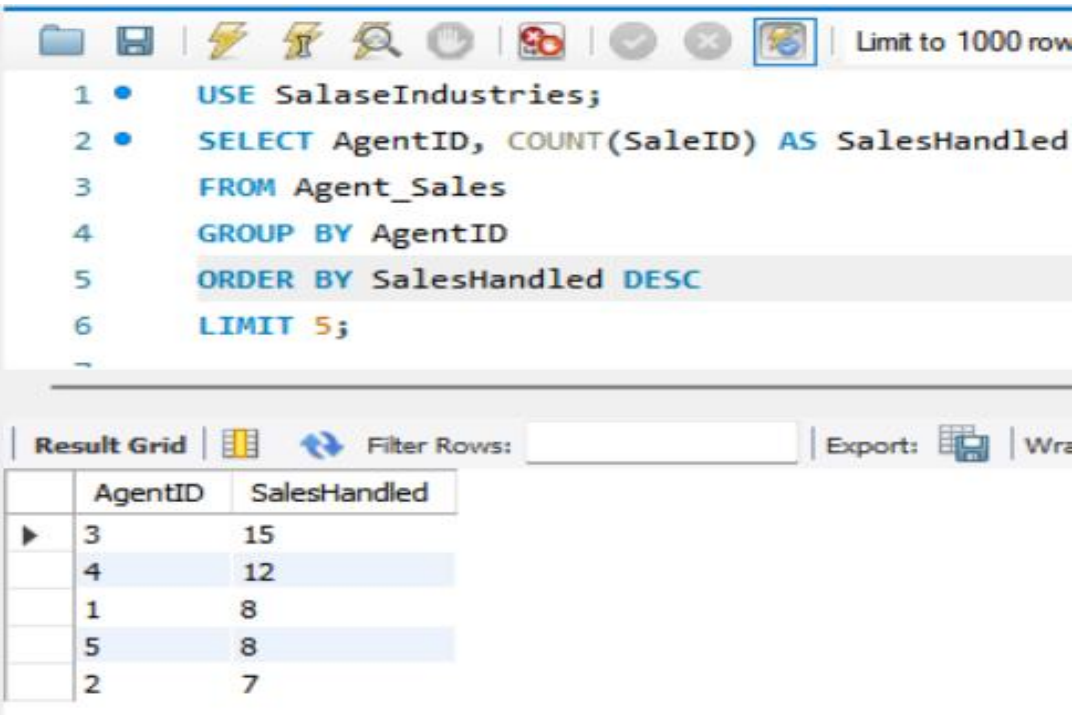


```
USE SalaseIndustries;
SELECT CustomerID, COUNT(SaleID) AS PurchaseCount
FROM Sales
GROUP BY CustomerID
ORDER BY PurchaseCount DESC
LIMIT 3;
```

Result Grid

CustomerID	PurchaseCount
1	6
10	6
3	6

29. Show the 5 agents with the most number of sales.

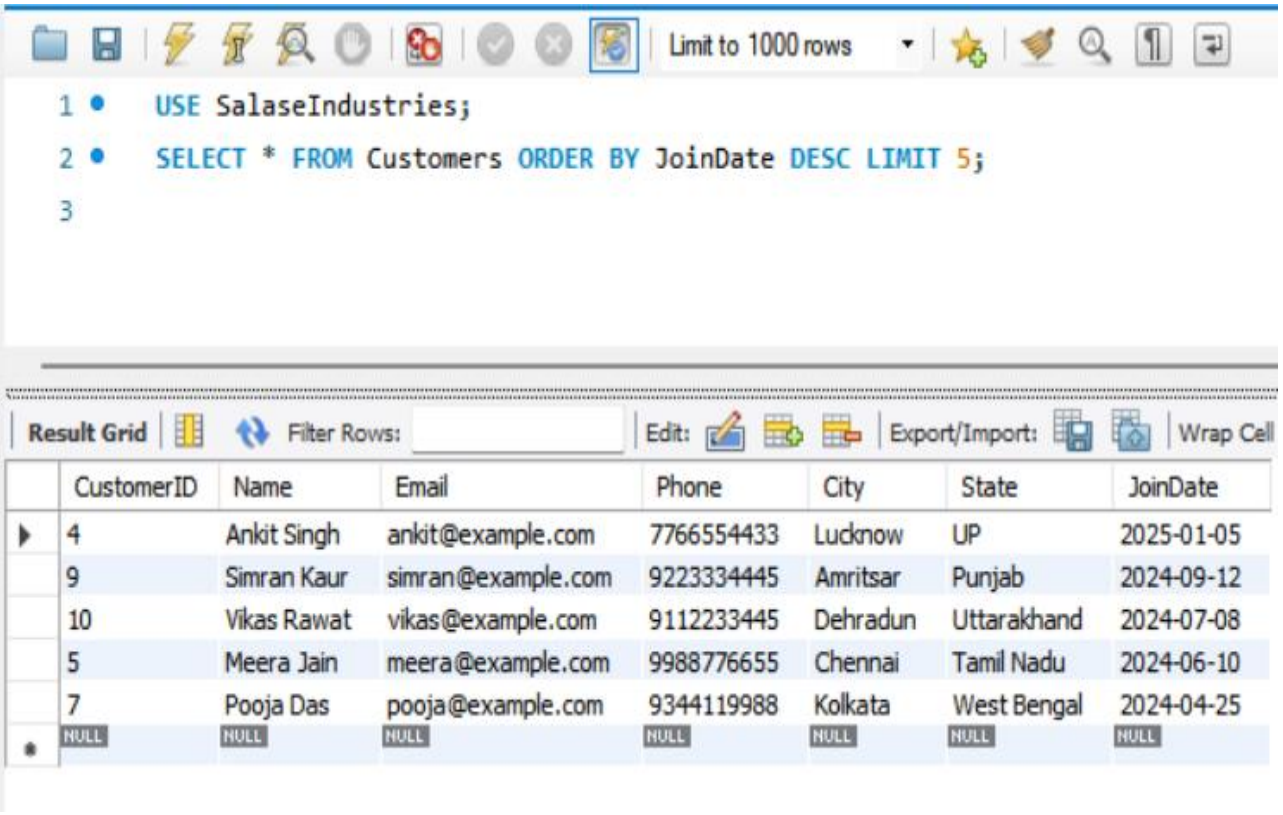


```
USE SalaseIndustries;
SELECT AgentID, COUNT(SaleID) AS SalesHandled
FROM Agent_Sales
GROUP BY AgentID
ORDER BY SalesHandled DESC
LIMIT 5;
```

Result Grid

AgentID	SalesHandled
3	15
4	12
1	8
5	8
2	7

30. List the latest 5 customers who joined.



```
USE SalaseIndustries;
SELECT * FROM Customers ORDER BY JoinDate DESC LIMIT 5;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

CustomerID	Name	Email	Phone	City	State	JoinDate
4	Ankit Singh	ankit@example.com	7766554433	Lucknow	UP	2025-01-05
9	Simran Kaur	simran@example.com	9223334445	Amritsar	Punjab	2024-09-12
10	Vikas Rawat	vikas@example.com	9112233445	Dehradun	Uttarakhand	2024-07-08
5	Meera Jain	meera@example.com	9988776655	Chennai	Tamil Nadu	2024-06-10
7	Pooja Das	pooja@example.com	9344119988	Kolkata	West Bengal	2024-04-25
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Conclusion

The **Sales and Analytics System – Salase Industries** project has been a meaningfu and practical experience in my data analytics journey. It allowed me to apply SQL skills in a real-world business context by designing a relational database system to track and analyze sales, customers, products, and agents for an FMCG company. By building a structured database and executing **30 insightful SQL queries**, I was able to answer key business questions related to revenue generation, product performance, customer spending patterns, stock levels, and sales agent efficiency. The project helped me understand how different parts of a business are connected through data and how SQL can be used to uncover trends and drive decisions.
Key skills practiced included:

- Writing advanced SQL queries using **JOIN**, **GROUP BY**, **HAVING**, **ORDER BY**, and **LIMIT**
- Understanding **one-to-many** and **many-to-many** relationships in databases
- Drawing **actionable insights** from raw data

This project strengthened my ability to think analytically, structure queries with purpose, and approach data problems from a business perspective. It also highlighted the importance of clean data models and accurate reporting in industries like FMCG where data-driven decisions are crucial.
I'm thankful to my mentors **Mr. Aman Chauhan** and **Mr. Vijay Narayan Singh** for their constant support and valuable guidance throughout this project.
Overall, this project laid a solid foundation for understanding how structured data systems support real business analytics, and it has significantly boosted my confidence and practical experience as a data analyst.

Submitted by:
TANVI AGGARWAL
BCA 2ND YEAR 4TH SEM
Tecnia Institute of Advanced Studies ,GGSIPU

End of Report